

conversation history, meeting notes, policy excerpts, and document passages. The risk is that summarisation may omit details, so it should be used with a similarity check or human review in high-stakes workflows.

Figure 2 shows how a practical open source framework can be designed as a modular Python package.

Each module should perform one responsibility, making the framework easier to test, extend, and integrate with existing applications. The architecture may include a prompt parser, rule engine, semantic deduplicator, token counter, optimization pipeline, evaluation module, and command-line interface.

Module	Purpose
Prompt parser	Splits the input prompt into sections, sentences, protected blocks, and candidate optimization units.
Rule optimizer	Removes filler phrases, redundant wording, repeated constraints, and unnecessary formatting noise.
Semantic deduplicator	Detects near-duplicate sentences or chunks using embeddings or similarity scoring.
Token counter	Calculates tokens before and after optimization using a model-specific tokenizer.
Safety validator	Checks whether required intent, constraints, and output format are still present.
CLI and API layer	Allows developers to use the optimizer from the command line, Python code, or a web service.

An example of a Python implementation

The following simplified implementation demonstrates the core idea. It is not a complete production framework, but it shows how a pipeline can remove filler phrases, normalise whitespace, deduplicate sentences, and report token savings.

```
import re
import tiktoken

class TokenCounter:
    def __init__(self, model="gpt-4o-mini"):
        self.encoding = tiktoken.encoding_for_model(model)

    def count(self, text: str) -> int:
        return len(self.encoding.encode(text))

class PromptOptimizer:
    FILLERS = [
        r"\bcould you please\b",
```

```
        r"\bplease make sure to\b",
        r"\bi would like you to\b",
        r"\bit would be great if you could\b",
        r"\bkindly\b"
    ]

    def __init__(self, model="gpt-4o-mini"):
        self.counter = TokenCounter(model)

    def remove_fillers(self, text: str) -> str:
        optimized = text
        for pattern in self.FILLERS:
            optimized = re.sub(pattern, "", optimized,
                               flags=re.IGNORECASE)
        return optimized

    def normalize_spaces(self, text: str) -> str:
        text = re.sub(r"\s+", " ", text)
        text = re.sub(r"\s+([.,;:])", r"\1", text)
        return text.strip()

    def deduplicate_sentences(self, text: str) -> str:
        sentences = re.split(r"(?<[!?!])\s+", text)
        seen = set()
        unique = []
        for sentence in sentences:
            key = re.sub(r"\W+", "", sentence.lower())
            if key and key not in seen:
                seen.add(key)
                unique.append(sentence)
        return " ".join(unique)

    def optimize(self, prompt: str) -> dict:
        before = self.counter.count(prompt)
        text = self.remove_fillers(prompt)
        text = self.deduplicate_sentences(text)
        text = self.normalize_spaces(text)
        after = self.counter.count(text)
        return {
            "original_prompt": prompt,
            "optimized_prompt": text,
            "tokens_before": before,
            "tokens_after": after,
            "tokens_saved": before - after,
            "saving_percent": round(((before - after) /
                                     before) * 100, 2) if before else 0
        }
```

Testing the optimizer with a sample prompt

Consider the following verbose prompt:

Could you please carefully analyse the customer complaint below and make sure to identify the main issue, urgency level, and recommended next action. I would like you to provide the